

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
import scipy
import statsmodels

import matplotlib as mpl
import matplotlib.pyplot as plt
import matplotlib.lines as mlines

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [pd, np, scipy, statsmodels, mpl, sns]:
    print(i.__name__, i.__version__)

from scipy.stats import norm
```

```
pandas 2.1.4
numpy 1.26.4
scipy 1.11.4
statsmodels 0.14.0
matplotlib 3.8.0
seaborn 0.12.2
```

```
In [2]: Image('107.png')
```

Out[2]:

2-8 분산 분석(ANOVA, Analysis of Variance)

서로 다른 범주에 속한 모집단의 평균 $\mu_0, \mu_1, \dots, \mu_k$ 동일한지 파악합니다.

$$H_0: \mu_0 = \mu_1 = \dots = \mu_k$$

$H_1: \mu_0, \mu_1, \dots, \mu_k$ 에서 동일하지 않는 것이 존재한다.

이 때 범주형 변수를 요인(factor)라고 합니다.

분산분석은

범주형을 가변수화 하여 모집단의 평균을 추정하는 선형 회귀 모델의 유의미성을 검정하는 과정과 유사합니다.

선형 회귀 모델에서 다루었던 제곱합의 성질을 통해, 유도했던 F-test를 적용한 것입니다.

$$SST = SSR + SSE$$

$$F = \frac{SSR/(p-1)}{SSE/(n-p)} \sim F(p-1, n-p), p: \text{회귀 모델의 모수의 수}, n: \text{표본수}$$

이 가설 검정에 응용한 것입니다.

그리고, 분산분석은 선형 회귀 모델의 가정을 따라갑니다.

- 서로 다른 범주에 모집단들의 분산은 동일 하다. (오차의 분산은 독립 변수와 상관없이 일정해야 합니다.)
- 서로 다른 범주에 모집단은 정규 분포를 따릅니다. (오차는 정규분포를 따라야 합니다.)

1. 일원분산분석 (Oneway ANOVA)

단일 요인에 대한 분산분석입니다.

데이터셋을 만들어봅니다.

```
In [2]: '''
코드는 다음을 수행합니다:

1. 평균(`mu`)과 표준편차(`sig`)가 주어진 정규분포를 따르는
   데이터를 생성합니다.
   데이터는 세 개의 그룹(`'A'`, `'B'`, `'C'`)으로 구성됩니다.
2. 각 그룹별로 다른 샘플 크기(`n`)가 설정되어 있습니다.
3. `norm.rvs()` 함수를 사용하여 정규 분포에서 무작위 표본을 생성합니다.
   평균은 `mu[i]`, 표준편차는 `sig`, 샘플 크기는 `n[i]`로 설정되어 있습니다.
4. 이러한 표본 데이터를 데이터프레임으로 변환하고, `'X'` 열에 저장합니다.
   또한 각 표본이 속한 그룹을 나타내는 `'factor'` 열을 추가합니다.
5. 이를 각 그룹별로 반복하여 생성하고,
   생성된 데이터프레임을 `pd.concat()` 함수를 사용하여
   수직으로 연결하여 하나의 데이터프레임으로 병합합니다.
6. 결과적으로 생성된 데이터프레임은 `df_ind`에 할당됩니다.

이 코드를 실행하면 세 개의 그룹으로 나뉘어진 정규분포를 따르는
데이터가 생성되며, 각 그룹별로 다른 샘플 크기가 적용됩니다.
'''

mu = {'A': 0.3, 'B': 0.3, 'C': 0.3}
sig = 0.2
n = {'A': 30, 'B': 25, 'C': 35}

df_ind = pd.concat([
    pd.DataFrame(norm.rvs(loc=mu[i], scale=sig, size=n[i],
                          random_state=123),
                  columns=['X']).assign(factor=i)
    for i in ['A', 'B', 'C']
], axis=0)
```

```
In [3]: mu = {'A': 0.3, 'B': 0.5, 'C': 0.7}
sig = 0.2
n = {'A': 30, 'B': 25, 'C': 35}

df_dep = pd.concat([
    pd.DataFrame(norm.rvs(loc=mu[i], scale=sig, size=n[i],
                          random_state=123),
                  columns=['X']).assign(factor=i)
```

```
for i in ['A', 'B', 'C']
], axis=0)
```

In [4]:

'''

코드는 두 개의 서브플롯을 포함하는 그림을 생성하고,
각각의 서브플롯에 대해 다음을 수행합니다:

1. Independent case: 독립적인 경우

- `sns.swarmplot()` 함수를 사용하여 `df_ind` 데이터프레임에서 각 그룹별로 'factor'에 따른 'X' 값의 분포를 산점도로 표시합니다.
- 각 그룹의 평균을 나타내기 위해 각 그룹의 'X' 값의 평균을 계산하고, 이를 빨간색 선으로 표시합니다.
- 이를 통해 각 그룹의 평균이 어떻게 분포되어 있는지를 확인할 수 있습니다.

2. Dependent case: 종속적인 경우

- `sns.swarmplot()` 함수를 사용하여 `df_dep` 데이터프레임에서 각 그룹별로 'factor'에 따른 'X' 값의 분포를 산점도로 표시합니다.
- 각 그룹의 평균을 나타내기 위해 각 그룹의 'X' 값의 평균을 계산하고, 이를 빨간색 선으로 표시합니다.
- 이번에는 각 그룹 간에 'X' 값의 평균이 서로 다르게 설정되어 있어, 이를 시각적으로 확인할 수 있습니다.

그림은 각 서브플롯에서 제목을 설정하고, 각 서브플롯의 레이아웃을 조정하여 겹치지 않도록 만듭니다.

이를 통해 독립적인 경우와 종속적인 경우에서 'factor'에 따른 'X' 값의 분포 및 평균을 시각적으로 비교할 수 있습니다.

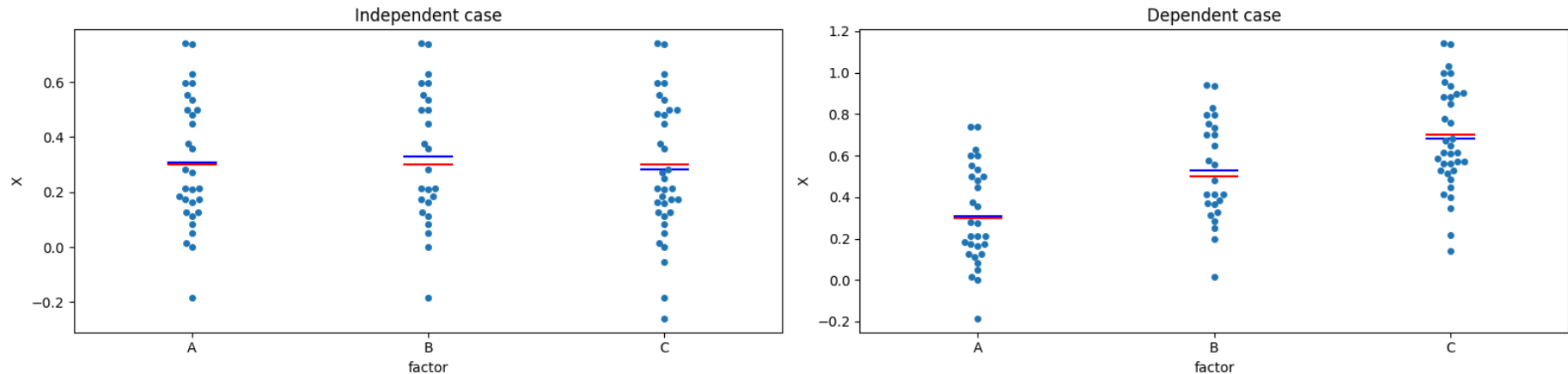
'''

```
fig, axes = plt.subplots(1, 2, figsize=(16, 4))
axes[0].set_title('Independent case')
sns.swarmplot(data=df_ind, x='factor', y='X', ax=axes[0])
for i, c in enumerate(['A', 'B', 'C']):
    x_bar = df_ind.loc[df_ind['factor'] == c, 'X'].mean()
    axes[0].add_line(mlines.Line2D([-0.1 + i, 0.1 + i], [0.3, 0.3], c='r'))
    axes[0].add_line(mlines.Line2D([-0.1 + i, 0.1 + i], [x_bar, x_bar], c='b'))

axes[1].set_title('Dependent case')
sns.swarmplot(data=df_dep, x='factor', y='X', ax=axes[1])
for i, c in enumerate(['A', 'B', 'C']):
    x_bar = df_dep.loc[df_dep['factor'] == c, 'X'].mean()
```

```
axes[1].add_line(mlines.Line2D([-0.1 + i, 0.1 + i], [0.3 + 0.2 * i, 0.3 + 0.2 * i], c='r'))
axes[1].add_line(mlines.Line2D([-0.1 + i, 0.1 + i], [x_bar, x_bar], c='b'))
```

```
plt.tight_layout()
plt.show()
```



[Ex.1]

df_ind에서 factor 에 대한 X의 모집단에 대한 분산분석을 합니다.

In [3]: `Image('108.png')`

Out[3]: statsmodel를 통한 방법입니다.

statsmodels.formula.api.ols → statsmodels.formula.api.anova_lm

※ ols의 formula(patsy 식) 범주형 변수의 가변수화는 C(변수명)로 표현하면됩니다.

```
In [5]: from statsmodels.formula.api import ols
from statsmodels.api import stats
lm = ols('X ~ C(factor)', data=df_ind).fit()
```

```
df_anova = stats.anova_lm(lm)
df_anova
```

```
Out[5]:
```

	df	sum_sq	mean_sq	F	PR(>F)
C(factor)	2.0	0.033030	0.016515	0.275669	0.759725
Residual	87.0	5.212032	0.059908	NaN	NaN

```
In [6]: print("검정통계량(F): {}, pvalue: {}".format(df_anova.loc['C(factor)', 'F'],
df_anova.loc['C(factor)', 'PR(>F)']))
```

검정통계량(F): 0.2756692188624851, pvalue: 0.7597245063787699

```
In [7]: # 일원분산분석은 선형회귀 모델의 F-test와도 동일한 결과를 냅니다.
lm.summary()
```

Out[7]:

OLS Regression Results

Dep. Variable:	X	R-squared:	0.006
Model:	OLS	Adj. R-squared:	-0.017
Method:	Least Squares	F-statistic:	0.2757
Date:	Mon, 04 Mar 2024	Prob (F-statistic):	0.760
Time:	18:40:14	Log-Likelihood:	0.49333
No. Observations:	90	AIC:	5.013
Df Residuals:	87	BIC:	12.51
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.3089	0.045	6.913	0.000	0.220	0.398
C(factor)[T.B]	0.0191	0.066	0.288	0.774	-0.113	0.151
C(factor)[T.C]	-0.0276	0.061	-0.452	0.652	-0.149	0.093

Omnibus:	4.137	Durbin-Watson:	1.980
Prob(Omnibus):	0.126	Jarque-Bera (JB):	2.156
Skew:	0.037	Prob(JB):	0.340
Kurtosis:	2.245	Cond. No.	3.76

Warnings:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

scipy.stats.f_oneway 로도 가능합니다.

```
In [8]: from scipy.stats import f_oneway
f_oneway(*df_ind.groupby('factor')['X'].agg(list))
```

```
Out[8]: F_onewayResult(statistic=0.27566921886248474, pvalue=0.7597245063787699)
```

```
In [4]: Image('109.png')
```

```
Out[4]:  $SST = SSR + SSE$ 
```

$$\sum_{i \in factor} \sum_{j=1}^{n_i} (y_{ij} - \bar{y})^2 = \sum_{i \in factor} \sum_{j=1}^{n_i} (\bar{y}_i - \bar{y})^2 + \sum_{i \in factor} \sum_{j=1}^{n_i} (y_{ij} - \bar{y}_i)^2$$

직접 구해봅니다.

```
In [9]: '''
코드는 분산 분석(ANOVA)을 수행하여
독립 변수(factor)에 따른 종속 변수(X)의 변화를 분석합니다.

1. `ss(v)` 함수는 주어진 벡터 `v`의 제곱의 합을 계산합니다.
   이 함수는 나중에 분산, 제곱합을 계산할 때 사용됩니다.
2. `sst`:
   총 변동(total variation) 또는 총 제곱합(total sum of squares, SST)을
   계산합니다. 이는 전체 데이터의 제곱합에서 평균을 뺀 값의
   제곱의 합입니다.
3. `ssr`:
   회귀 변동(regression variation) 또는
   회귀 제곱합(regression sum of squares, SSR)을 계산합니다.
   이는 각 그룹의 평균값과 전체 데이터의 평균값의 차이를
   제곱하여 합산한 값입니다.
4. `sse`:
   잔차 변동(residual variation) 또는
   잔차 제곱합(residual sum of squares, SSE)을 계산합니다.
   이는 각 데이터의 실제 값과 해당 그룹의 평균값의 차이를
   제곱하여 합산한 값입니다.
5. `df_ssr`: SSR의 자유도(degree of freedom)를 계산합니다.
   이는 그룹의 수에서 1을 뺀 값입니다.
6. `df_sse`:
```


SSE의 자유도를 계산합니다.

이는 전체 데이터의 수에서 그룹의 수를 뺀 값입니다.

7. `F`:

F-통계량(F-statistic)을 계산합니다.

이는 SSR의 자유도에 대한 SSR의 변동과 SSE의 자유도에 대한

SSE의 변동의 비율을 계산한 값으로,

그룹 간의 분산과 그룹 내의 분산을 비교하여

그룹 간의 변동이 통계적으로 유의한지를 확인합니다.

따라서 코드는 ANOVA를 수행하여 그룹 간의 평균 차이가 통계적으로 유의한지를 검정합니다.

F-통계량이 유의수준에서 유의하다면,

적어도 하나의 그룹의 평균이 다른 그룹들과 유의미하게 다르다는 것을 의미합니다.

'''

```
from scipy.stats import f
```

```
def ss(v):
    return (v ** 2).sum()
```

```
sst = ss(df_ind['X'] - df_ind['X'].mean())
```

```
ssr = ss(df_ind.groupby('factor')['X']
        .transform('mean') - df_ind['X'].mean())
```

```
sse = ss(df_ind['X'] - df_ind.groupby('factor')['X']
        .transform('mean'))
```

```
df_ssr = df_ind['factor'].nunique() - 1
```

```
df_sse = len(df_ind) - df_ind['factor'].nunique()
```

```
F = (ssr / df_ssr) / (sse / df_sse)
```

```
print("검정통계량(F): {}, pvalue: {}".format(F, f.sf(F, df_ssr, df_sse)))
```

```
sst, ssr, sse
```

검정통계량(F): 0.275669218862485, pvalue: 0.7597245063787699

Out[9]: (5.2450613652543945, 0.03302980845858672, 5.212031556795809)

[Ex.2]

df_dep에서 categ 에 대한 X의 모집단에 대한 분산분석을 합니다.

분산 분석에 앞서 categ의 범주에 따라 모집단이 정규분포를 따르는지

Kolmogorov-smirnof 검정으로 확인하고,

Bartlett 검정을 통해 서로 등분산인지 확인합니다.

In [10]:

```
'''
코드는 그룹별로 샘플이 종속적인 경우에 대해
Kolmogorov-Smirnov(KS) 검정을 수행하여 각 그룹의 데이터가
정규 분포를 따르는지를 확인하는 작업을 수행합니다.

1. `df_dep.groupby('factor')['X']`:
   'factor' 열을 기준으로 데이터를 그룹화하고,
   'X' 열에 대해 그룹별로 작업을 수행합니다.
2. `apply(lambda x: pd.Series(kstest(zscore(x, ddof=1),
   norm.cdf), index=['stat', 'pvalue']))`:
   각 그룹 내에서 다음 작업을 수행합니다.
   - `zscore(x, ddof=1)`:
     각 그룹의 'X' 데이터에 대해 Z-score를 계산합니다.
     이는 평균을 0으로, 표준 편차를 1로 만들어주는
     표준화 작업입니다.
   - `kstest()`:
     계산된 Z-score 값을 사용하여 KS 검정을 수행합니다.
     KS 검정은 주어진 데이터가 특정 분포(여기서는 정규 분포)를
     따르는지를 확인하는 검정 방법 중 하나입니다.
     이 함수는 KS 검정의 검정 통계량(statistic)과
     p-value를 반환합니다.
   - `pd.Series(..., index=['stat', 'pvalue'])`:
     KS 검정 결과를 pandas Series로 변환하고,
     해당 인덱스를 'stat'과 'pvalue'로 지정합니다.
3. `.unstack()`: 결과를 데이터프레임으로 변환합니다.
   이때 그룹별로 계산된 KS 검정의 검정 통계량(statistic)과
   p-value가 각각 열을 형성하도록 데이터를 재구성합니다.

따라서 코드는 각 그룹의 데이터가 정규 분포를 따르는지를 확인하기 위해
KS 검정을 수행하고, 검정 통계량(statistic)과
p-value를 출력하는 작업을 수행합니다.
'''
```

```

from scipy.stats import kstest, zscore
df_dep.groupby('factor')['X'].apply(
    lambda x: pd.Series(kstest(zscore(x, ddof=1), norm.cdf),
                        index=['stat', 'pvalue'])
).unstack()

```

Out[10]:

	stat	pvalue
factor		
A	0.155033	0.423738
B	0.157950	0.510746
C	0.120307	0.647730

```

In [11]: from scipy.stats import bartlett
bartlett(*df_dep.groupby('factor')['X'].agg(list))

```

Out[11]: BartlettResult(statistic=0.08114650401701767, pvalue=0.9602388225127009)

```

In [12]: from scipy.stats import f_oneway
f_oneway(*df_dep.groupby('factor')['X'].agg(list))

```

Out[12]: F_onewayResult(statistic=18.753725760815882, pvalue=1.6907914024262457e-07)

2. 이원분산분석 (Twoway ANOVA)

두 개의 요인에 대한 분산분석입니다.

상호작용(교호작용)

두 요인이 작용하여 종속 변수에 미치는 영향이 개별적인 영향의 합으로 설명되지 않는 현상

※ 교호작용은 교호항 마다 두 개이상의 관측치가 있어야 합니다.

교호작용이 존재하는 데이터셋을 만들어 봅니다.

In [13]:

```
'''
코드는 두 개의 요인(factor)을 고려한
실험 설계를 기반으로 데이터를 생성합니다.

1. `mu`: 두 개의 요인(factor)에 대한 평균값을 설정합니다.
   딕셔너리 형태로 요인(factor)의 조합에 대한
   평균값이 지정됩니다.
   예를 들어, ('A', '1')은 첫 번째 요인이 'A'이고
   두 번째 요인이 '1'인 경우의 평균값을 의미합니다.
2. `sig`:
   실험에서 사용할 표준편차를 설정합니다.
3. `n`:
   두 개의 요인(factor)에 대한 샘플 크기를 설정합니다.
   딕셔너리 형태로 요인(factor)의 조합에 대한
   샘플 크기가 지정됩니다.
4. `pd.concat(..., axis=0)`:
   itertools의 `product()` 함수를 사용하여
   두 개의 요인(factor)의 모든 조합에 대해
   반복문을 실행하고, 각 조합에 대한 데이터를 생성합니다.
   - `norm.rvs()`:
     정규 분포에서 무작위 표본을 생성합니다.
     평균은 `mu[(i, j)]`, 표준편차는 `sig`,
     샘플 크기는 `n[(i, j)]`로 설정됩니다.
     각 요인(factor)의 조합마다
     평균값이 다르게 설정되어 있습니다.
   - 생성된 데이터는 'X' 열에 저장됩니다.
   - 각 요인(factor)의 값은 'factor_1'과
     'factor_2' 열에 저장됩니다.

결과적으로 코드는 두 개의 요인(factor)을 고려한
실험 설계에 따라 데이터를 생성하고,
각 데이터를 하나의 데이터프레임으로 병합합니다.
이렇게 생성된 데이터는 두 개의 요인(factor)에 따른
실험 결과를 반영하며, 각 요인(factor)의 조합별로
다른 평균값을 가집니다.
'''

import itertools
mu = {
```

```

    ('A', '1'): 0.2, ('A', '2') : 0.3, ('A', '3') : 0.4,
    ('B', '1'): 0.3, ('B', '2') : 0.4, ('B', '3') : 0.2
}
sig = 0.2
n = {
    ('A', '1'): 15, ('A', '2') : 25, ('A', '3') : 20,
    ('B', '1'): 20, ('B', '2') : 30, ('B', '3') : 25
}

df_two = pd.concat([
    pd.DataFrame(norm.rvs(loc=mu[(i, j)], scale=sig,
                          size=n[(i, j)], random_state=123),
                  columns=['X']).assign(factor_1=i,
                                       factor_2=j)
    for i,j in itertools.product(['A', 'B'], ['1', '2', '3'])
], axis=0)

```

교호작용이 있다면,

factor_1와 factor_2 로 구분하여 평균에 대한 그래프를

factor_2의 항목을 x축에 종속 변수의 평균을 y축에, 그리고 factor_1으로 선을 구분하여,

즉 아래와 같이 직선 그래프로 출력하면,

factor_1이 A일 때와 factor_1이 B일 때, factor_2 별 성향이 다르다는 것을 확인 할 수 있습니다.

이렇다면, factor_1과 factor_2의 개별의 평균만으로 설명하기 어렵게 됩니다.

```

In [14]: '''
코드는 두 개의 요인(factor)에 따른
실험 결과를 시각화하는 작업을 수행합니다.

1. `df_two.groupby(['factor_1', 'factor_2'],
                    as_index=False)['X'].mean()`:
먼저 데이터프레임을 'factor_1'과 'factor_2' 열을 기준으로
그룹화하고, 각 그룹별로 'X' 열의 평균값을 계산합니다.
이를 통해 각 요인(factor)의 조합에 따른 평균 'X' 값을 얻습니다.
'''

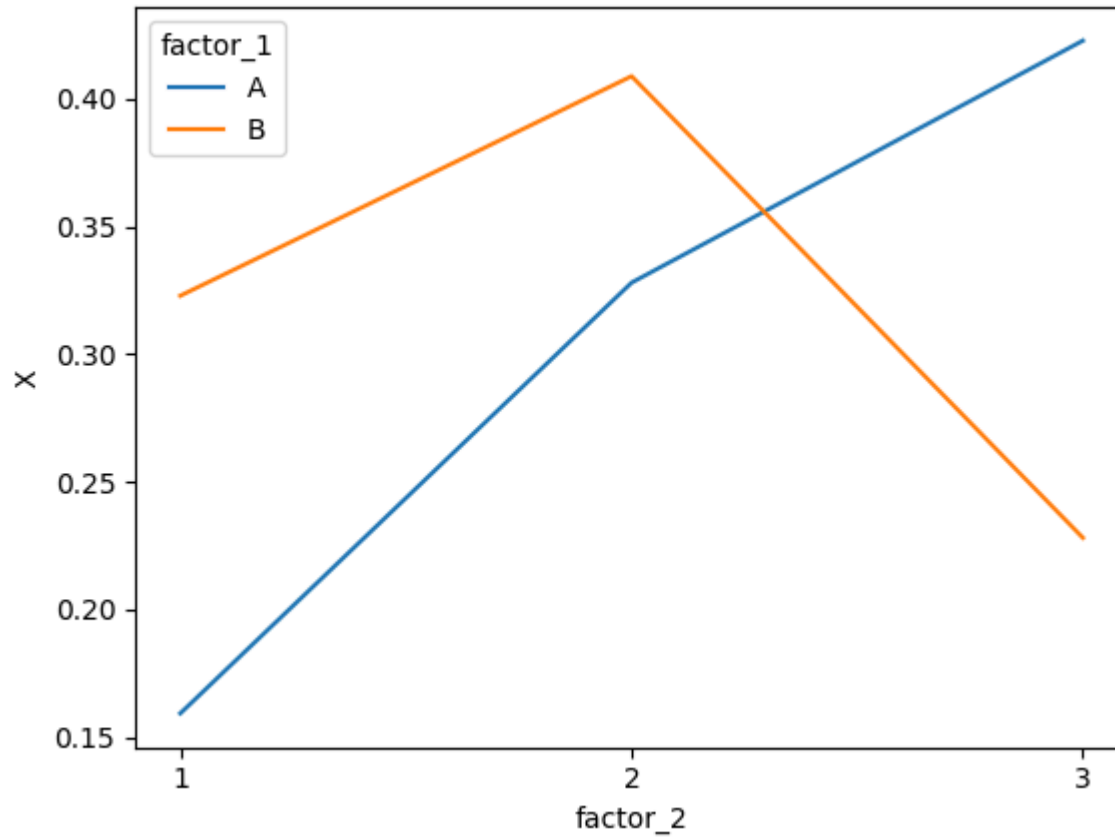
```

```
2. `.pipe(lambda x: sns.lineplot(data=x, x='factor_2',
                                y='X', hue='factor_1'))`:
    이전에 계산된 평균값을 시각화합니다.
- `sns.lineplot()`:
    seaborn 패키지의 `lineplot()` 함수를
    사용하여 선 그래프를 생성합니다.
- `data=x`:
    시각화할 데이터는 이전에 계산된
    평균값인 `x`를 사용합니다.
- `x='factor_2'`, `y='X'`:
    x축에는 'factor_2' 열을, y축에는 'X' 열의 값을
    사용합니다. 즉, 'factor_2' 값에 따른
    'X' 값의 변화를 보여주는 선 그래프를 생성합니다.
- `hue='factor_1'`:
    선 그래프의 색상을 'factor_1' 열의 값에 따라
    다르게 설정합니다. 즉, 'factor_1' 값에 따라
    선 그래프의 색상이 달라지며,
    각 선은 'factor_1' 값에 해당하는 그룹을 나타냅니다.

따라서 위 코드는 두 개의 요인(factor)에 따른 실험 결과를
시각화하여, 'factor_2' 값에 따른 'X' 값의 변화를
그룹별로 나타내는 선 그래프를 생성합니다.
선의 색상은 'factor_1' 값에 따라 다르게 표현되어,
각 요인(factor)에 대한 변화를 쉽게 비교할 수 있습니다.
'''

df_two.groupby(['factor_1', 'factor_2'], as_index=False)['X'].mean().pipe(
    lambda x: sns.lineplot(data=x, x='factor_2', y='X', hue='factor_1')
)
```

Out[14]: <AxesSubplot:xlabel='factor_2', ylabel='X'>

**[Ex.3]**

df_two에서 X에 대한 factor1, factor2의 이원분산분석을 해봅니다.

statsmodel를 통한 방법입니다.

※ ols의 formula(patsy 식) 두 범주형 변수의

교호항 가변수화는 C(변수명1):C(변수명2)로 표현하면됩니다.

```
In [15]: '''
코드는 두 개의 요인(factor)에 대한
이차원 분산분석(ANOVA)을 수행합니다.

1. `ols('X ~ C(factor_1) + C(factor_2) + C(factor_1):C(factor_2)',
        data=df_two)` :
    이 코드는 'X'를 종속 변수로 하고, 'factor_1', 'factor_2',
    그리고 두 요인의 상호작용항인 'factor_1:factor_2'를
    설명 변수로 하는 선형 회귀 모델을 생성합니다.
    - `C()` 함수는 범주형 변수를 지정합니다.
    - `factor_1 * factor_2`는 'factor_1'과 'factor_2'의
      상호작용항을 나타냅니다.
2. `.fit()` :
    선형 회귀 모델을 데이터에 적합시킵니다.
    이를 통해 회귀 분석 결과를 얻을 수 있습니다.
3. `stats.anova_lm(lm)` :
    이 코드는 회귀 분석 결과를 이용하여
    분산 분석(ANOVA)을 수행합니다.
    - `anova_lm()` 함수는 선형 모델의 분산분석 결과를 계산합니다.
    - `lm`은 이전에 적합시킨 선형 회귀모델을 의미합니다.

따라서 코드는 두 개의 요인(factor)에 대한 이차원 분산분석을 수행하고,
각 요인의 주효과 및 상호작용 효과를 검정하여
모델의 유의성을 평가합니다.
이를 통해 요인 간의 차이 및 상호작용이
통계적으로 유의한지를 확인할 수 있습니다.
'''

'''

아래의 ANOVA 결과는 두 요인(factor) 및 그들의 상호작용에 대한
효과를 검정한 것입니다. 결과를 해석하는 방법은 다음과 같습니다:

1. C(factor_1):
    - F 값이 0.038022이며 p-value가 0.845706입니다.
    - 요인(factor_1)에 대한 F 값이 유의수준 0.05보다 작거나 같습니다.
    - 따라서 factor_1의 주효과는 통계적으로 유의하지 않습니다.
      즉, factor_1의 수준이 결과인 'X'에
      유의미한 영향을 미치지 않는 것으로 나타납니다.

2. C(factor_2):
```


- F 값이 2.597120이며 p-value가 0.078379입니다.
- 요인(factor_2)에 대한 F 값이 유의수준 0.05보다 작지 않지만, 0.1보다는 작습니다.
- 따라서 factor_2의 주효과는 유의하지 않지만, 유의수준이 0.1보다는 낮습니다.
따라서 factor_2의 효과가 약간의 통계적인 유의성을 보일 수 있습니다.

3. C(factor_1):C(factor_2) (상호작용):

- F 값이 6.177722이며 p-value가 0.002741입니다.
- 요인(factor_1)과 요인(factor_2)의 상호작용에 대한 F 값이 유의수준 0.05보다 작습니다.
- 따라서 factor_1과 factor_2 간의 상호작용 효과는 통계적으로 유의합니다.
즉, 요인(factor_1)의 효과가
요인(factor_2)의 수준에 따라 달라집니다.

4. Residual:

- 나머지 잔차에 대한 통계량입니다.
- sum_sq는 129.0, mean_sq는 7.698618,
F 값은 NaN이며 p-value도 NaN입니다.
- 잔차에 대한 F 값 및 p-value는 의미가 없습니다.
잔차는 모델로 설명되지 않은 변동성을 나타내므로
통계적으로 유의한 값이 아닙니다.

요약하면, 위 결과는 요인(factor) 간에
유의한 상호작용 효과가 있으며,
factor_1과 factor_2의 효과는 모두 유의하지 않지만,
factor_2의 효과가 약간의 유의성을 보일 수 있습니다.
...

```
lm = ols('X ~ C(factor_1) + C(factor_2) + C(factor_1):C(factor_2)',
         data=df_two).fit()
df_anova = stats.anova_lm(lm)
df_anova
```

Out[15]:

	df	sum_sq	mean_sq	F	PR(>F)
C(factor_1)	1.0	0.002269	0.002269	0.038022	0.845706
C(factor_2)	2.0	0.309988	0.154994	2.597120	0.078379
C(factor_1):C(factor_2)	2.0	0.737363	0.368682	6.177722	0.002741
Residual	129.0	7.698618	0.059679	NaN	NaN

In [5]: `Image('110.png')`

Out[5]: SSR 은 아래와 같이 SSR_{factor_1} , SSR_{factor_2} , $SSR_{factor_1, factor_2}$ 로 분해가 될 수 있습니다.

$$SST = SSR_{factor_1} + SSR_{factor_2} + SSR_{factor_1, factor_2} + SSE$$

n_1 : factor1의 요인 수, n_2 : factor2의 요인 수 라면,

교호항의 가변수를 추가 할 때에는,

각 범주형 변수에서 하나의 범주씩을 제외한 순서쌍으로 가변수를 만들어야 완전한 다중공선성을 피할 수 있습니다.

따라서 $(n_1 - 1)(n_2 - 1)$ 교호항에 대한 가변수가 추가됩니다.

SSR 의 자유도는 $n_1 n_2 - 1$ 입니다.

SSR_{factor_1} 의 자유도는 $n_1 - 1$, SSR_{factor_2} 의 자유도는 $n_2 - 1$ 이므로

$SSR_{factor_1, factor_2}$ 의 자유도는 $n_1 n_2 - 1 - n_1 + 1 - n_2 + 1 = (n_1 - 1)(n_2 - 1)$ 이 됩니다.

In [16]: `'''`
 코드는 분산 분석(ANOVA)을 수행하기 위해 필요한
 각 요인(factor) 및 상호작용에 대한
 제곱합(sum of squares)을 계산하는 함수를 정의하고 있습니다.

1. `ss()` 함수:
 - 이 함수는 주어진 값들의 제곱합(sum of squares)을 계산합니다.
 - 입력으로는 배열 또는 Series 형태의 데이터가 주어집니다.
 - 주어진 값들을 제곱하여 합산한 값을 반환합니다.
2. 주효과 및 상호작용에 대한 제곱합 계산:
 - `X_bar = df_two['X'].mean()`:
전체 평균을 계산합니다.
 - `sst = ss(df_two['X'] - X_bar)`:
전체 제곱합(Total Sum of Squares)을 계산합니다.
이는 전체 변동성을 나타냅니다.
 - `X_prd = df_two.groupby(['factor_1', 'factor_2'])['X'] \`
`.transform('mean')`:
각 요인(factor) 및 그들의 조합에 대한 평균값을 계산합니다.
이를 통해 예측값(predicted values)을 얻습니다.
 - `ssr = ss(X_prd - X_bar)`:
회귀 제곱합(Regression Sum of Squares)을 계산합니다.
이는 모델이 설명하는 변동성을 나타냅니다.
 - `ssr_1 = ss(df_two.groupby('factor_1')['X']`
`.transform('mean') - X_bar)`:
첫 번째 요인(factor_1)에 대한 회귀 제곱합을 계산합니다.
 - `ssr_2 = ss(df_two.groupby('factor_2')['X']`
`.transform('mean') - X_bar)`:
두 번째 요인(factor_2)에 대한 회귀 제곱합을 계산합니다.
 - `ssr_12 = ssr - ssr_1 - ssr_2`:
상호작용에 대한 회귀 제곱합을 계산합니다.
 - `sse = ss(df_two['X'] - X_prd)`:
잔차 제곱합(Residual Sum of Squares)을 계산합니다.
이는 모델로 설명되지 않는 변동성을 나타냅니다.

이렇게 계산된 제곱합들은 분산 분석표(ANOVA table)에서
주요한 지표로 사용되며, 요인 간의 효과와 상호작용에 대한
통계적 추론을 가능하게 합니다.

```
from scipy.stats import f
```

```
def ss(v):
    return (v ** 2).sum()
```

```
X_bar = df_two['X'].mean()
```

```

sst = ss(df_two['X'] - X_bar)

X_prd = df_two.groupby(['factor_1', 'factor_2'])['X'] \
        .transform('mean')
ssr = ss(X_prd - X_bar)
ssr_1 = ss(df_two.groupby('factor_1')['X'] \
           .transform('mean') - X_bar)
ssr_2 = ss(df_two.groupby('factor_2')['X'] \
           .transform('mean') - X_bar)
ssr_12 = ssr - ssr_1 - ssr_2

sse = ss(df_two['X'] - X_prd)

```

In [17]:

```

'''
코드는 상호작용 효과에 대한
F 통계량을 계산하는 과정을 나타냅니다.

1. `n1`과 `n2` 계산:
- `n1 = df_two['factor_1'].nunique()`:
  첫 번째 요인(factor_1)의 수준의 개수를 계산합니다.
- `n2 = df_two['factor_2'].nunique()`:
  두 번째 요인(factor_2)의 수준의 개수를 계산합니다.

2. 자유도 계산:
- `df_ssr_12 = (n1 - 1) * (n2 - 1)`:
  상호작용 항의 자유도를 계산합니다.
- `(n1 - 1)`은 첫 번째 요인의 자유도이며,
  `(n2 - 1)`은 두 번째 요인의 자유도입니다.
- 상호작용 항의 자유도는 각 요인의 수준의 개수에서
  1을 뺀 값들의 곱입니다.
- `df_sse = len(df_two) - n1 * n2`:
  잔차의 자유도를 계산합니다.
- 데이터프레임의 길이에서 각 요인의 수준의 개수를
  곱한 값을 뺀 것이 잔차의 자유도입니다.

3. F 통계량 계산:
- `F = (ssr_12 / df_ssr_12) / (sse / df_sse)`:
  상호작용 효과에 대한 F 통계량을 계산합니다.
- 상호작용 항의 평균제곱(제곱합을 자유도로 나눈 값)을
  상호작용 항의 자유도로 나누어주고,

```

- 잔차의 평균제곱을 잔차의 자유도로 나누어준 뒤,
그 두 값을 나눈 것입니다.
- 이렇게 계산된 F 통계량은 상호작용 항의
유의성을 평가하는 지표로 사용됩니다.

이렇게 계산된 F 통계량은 상호작용 효과에 대한
유의성을 평가하는 데 사용됩니다.
F 통계량이 유의수준을 넘어설 경우,
상호작용 효과가 통계적으로 유의하다고 할 수 있습니다.
'''

```
n1 = df_two['factor_1'].nunique()
n2 = df_two['factor_2'].nunique()
df_ssr_12 = (n1 - 1) * (n2 - 1)
df_sse = len(df_two) - n1 * n2
F = (ssr_12 / df_ssr_12) / (sse / df_sse)
F
```

Out[17]: 6.188013525470501

In []: